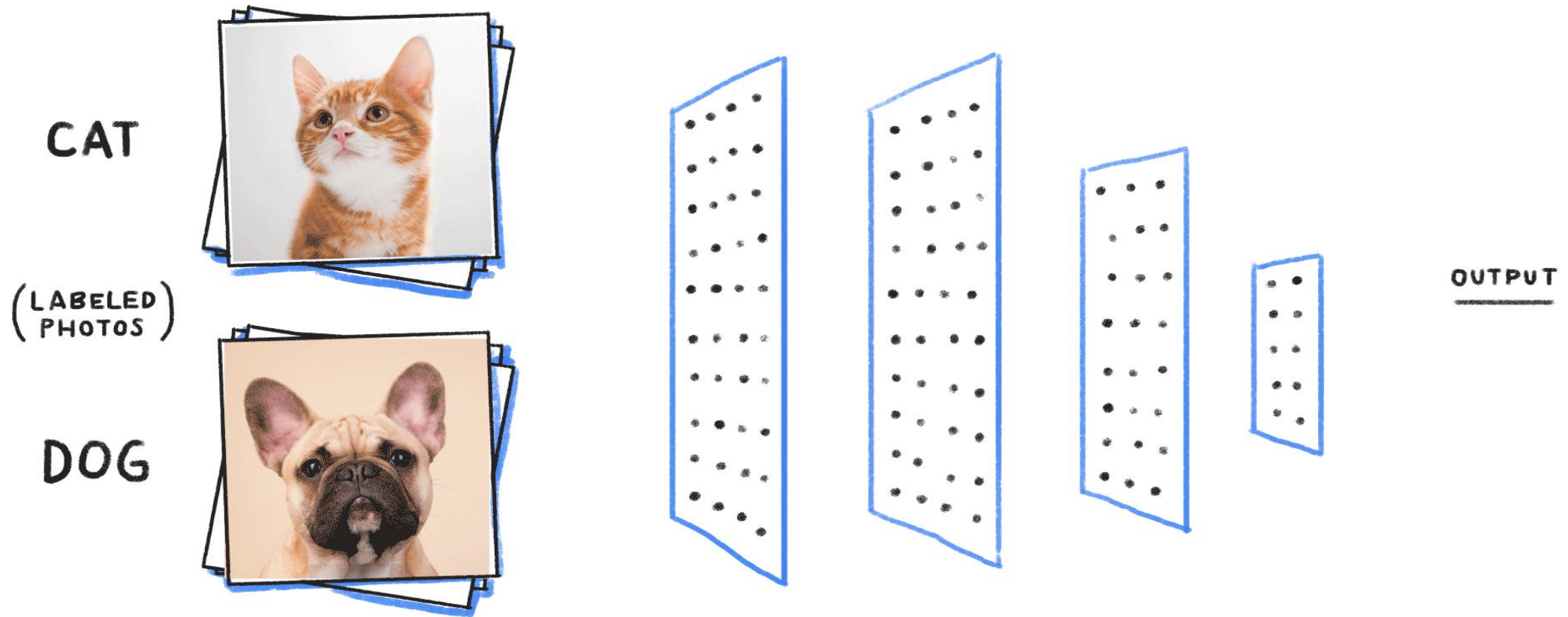


UNIT 5:- CONVOLUTIONAL NEURAL NETWORK



What is CNN?

- ▶ A Convolutional Neural Network (CNN) is a type of Deep Learning neural network architecture commonly used in Computer Vision.
 - ▶ Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.
- ▶ Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset.
 - ▶ For example visual datasets like images or videos where data patterns play an extensive role.

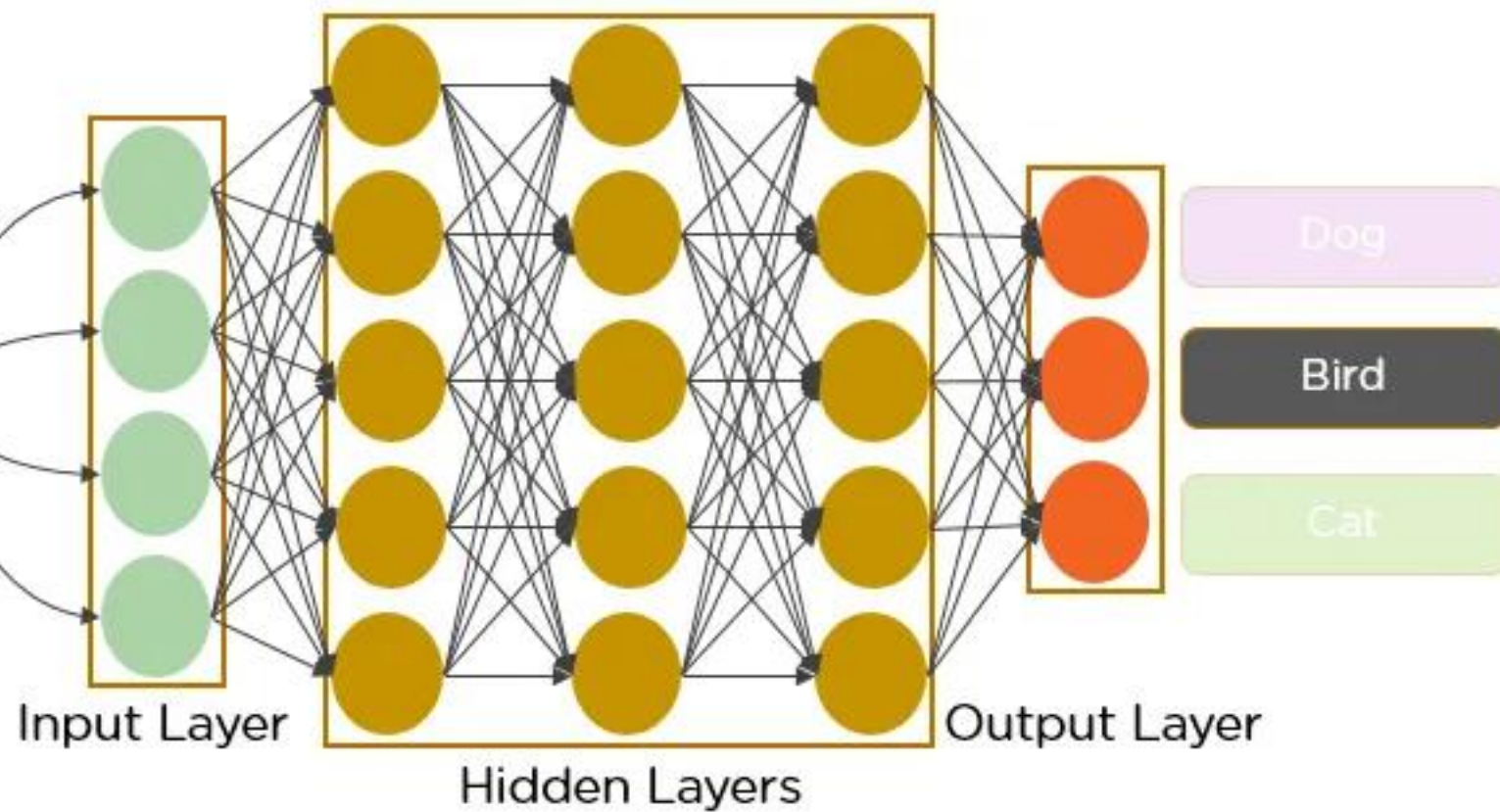
ANN in Machine Learning

- ▶ When it comes to Machine Learning, Artificial Neural Networks perform really well. Neural Networks are used in various datasets like images, audio, and text. Different types of Neural Networks are used for different purposes.
- ▶ for example--- for predicting the sequence of words we use Recurrent Neural Networks more precisely an LSTM, similarly for image classification we use Convolution Neural networks.

- When classifying images, traditional neural networks struggle because each pixel is treated as an independent feature, which misses critical patterns in how pixels interact. Some pixels combined together generate patterns which are not recognized if treated separately. So, we need some new way to overcome this problem.
- Imagine there's an image of a bird, and you want to identify whether it's really a bird or some other object. The first thing you do is feed the pixels of the image in the form of arrays to the input layer of the neural network (multi-layer networks used to classify things).
- A CNN consists of an input and an output layer, as well as multiple hidden layers. The hidden layers of a CNN typically consist of convolutional layers, pooling layers (that perform feature extraction from the image), fully connected layers (that identifies the object in the image) and normalization layers.



Pixels of image fed as input



Why should we use CNN?

Problem with Feedforward Neural Network

- Imagine you're working with the **MNIST dataset**, which is a collection of images of handwritten digits. Each image is small, just **28 x 28 pixels** in size, and it's black and white, so it has only **1 color channel** (meaning it only needs one layer to represent brightness values). When you feed this image to a neural network, each pixel is treated as a separate “input” or “neuron.” Since there are 28 rows and 28 columns of pixels, you have a total of **28 x 28 = 784 neurons**. For small images like this, that number is manageable.
- Now, think about a much larger image, say, **1000 x 1000 pixels**, which is closer to the size of a standard photo. If you try to handle each pixel separately as a neuron, you'd need **1,000 x 1,000 = 1,000,000 neurons** in the input layer. That's **a million neurons**! It's like going from a small audience to a huge stadium full of people — it's much more challenging to manage.

- CNNs make working with large images easier.
- Instead of treating each pixel as a separate neuron, CNNs use **filters** (also called **kernels**) to scan small sections, or patches, of the image at a time.
- This lets CNNs focus on **important features** in the image, like edges or textures, without needing to handle every pixel individually.
- CNNs combine these small patches into a simplified, lower-dimensional representation that keeps the essential details of the image.

For example:

1. Suppose we start with an image that's **224 x 224 pixels** with 3 color channels (RGB). If we fed it directly to a neural network without convolution, we'd need **$224 \times 224 \times 3 = 150,528$ neurons** in the input layer.
2. But with a CNN, we can apply several **convolutional filters** that gradually reduce the size of the image's data, capturing only the essential features.
3. By the time the image has passed through multiple convolutional layers, its data might be reduced to just **$1 \times 1 \times 1000$** .
4. This final representation only needs **1000 neurons** in the next layer, instead of 150,528.

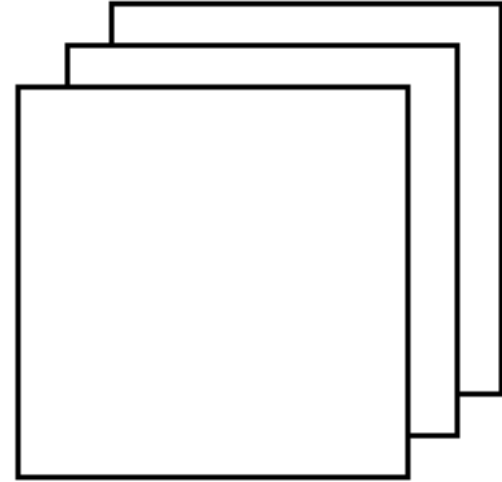
Why CNNs are Better than Traditional Networks for Images

- In regular neural networks, each pixel would need its own weight, making them **very large and inefficient** for images. For example, a **100 x 100 image** has 10,000 pixels, meaning each neuron in the next layer would need **10,000 weights** — a huge number of calculations!
- CNNs solve this by using **filters** that “share weights.” This means they scan the image in small sections, using the same small set of weights across the image. This approach:
- **Reduces the number of parameters** (weights) needed.
- **Allows the CNN to go deeper (adding more layers) to learn complex features without overwhelming computational power.**
- **By using these shared weights and small filters, CNNs capture important features without needing tons of calculations and memory, making them very efficient and powerful for image tasks.**

Input Image



2D image



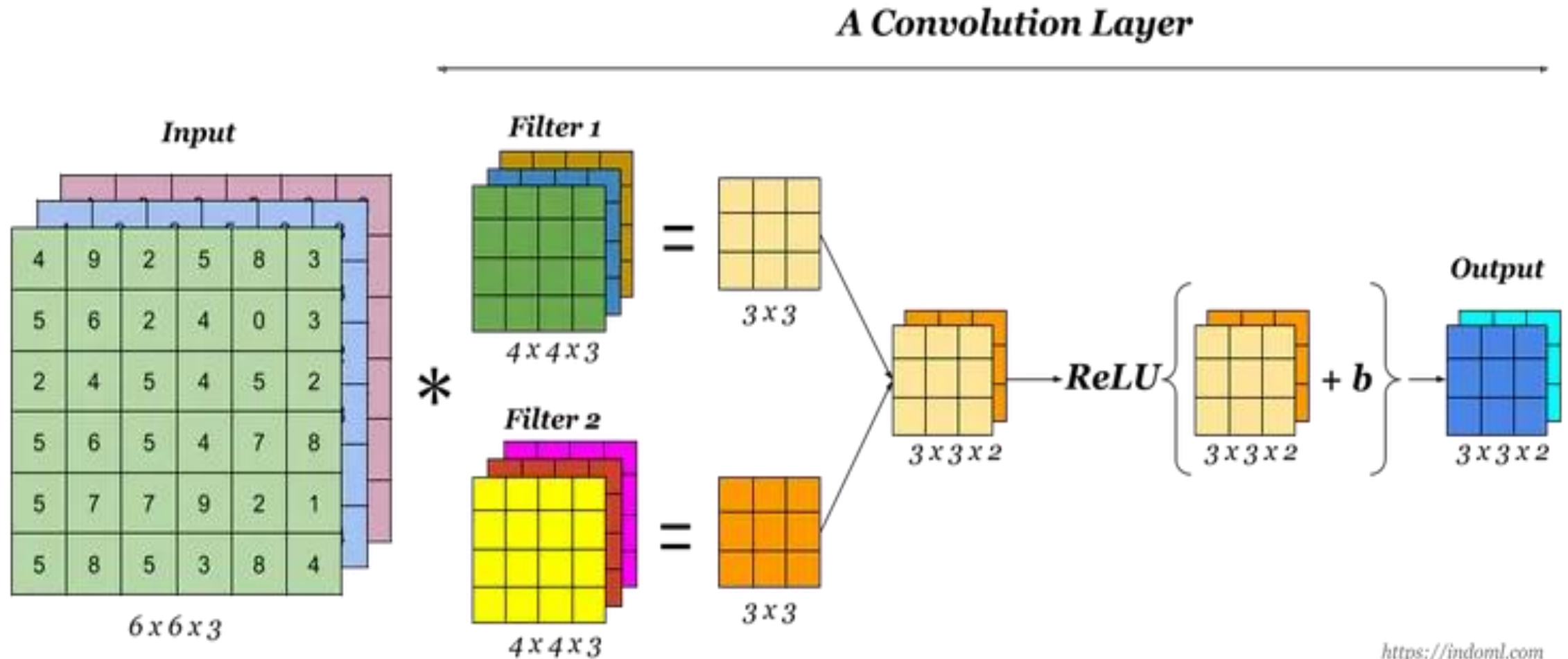
3D image

Greyscale VS RGB Image

First of all, the input image will be broken down into pixels.

If it is a black and white image, it will only have one layer and pixels will be interpreted as 2D array with the value from 0 to 255. If it is colored image, it will have 3 layers (red, green, blue) and will be interpreted as 3D array.

Important CNN Layers



1.Convolutional Layer

Think of the **convolutional layer** as the “eyes” of a CNN. It scans the image to find important features, like edges, shapes, or textures. When a feature is detected, the neurons in this layer produce a high response, which is called “activation.”

What is Convolution?

Convolution is like combining two things to make something new. In CNNs, the image is combined with small patterns, called **filters** (or **kernels**), which act as templates to find specific features in the image.

Imagine you’re looking at a small part of the image, like a **3 x 3 square** of pixels. To apply a filter:

- 1.Multiply each pixel in the square by a corresponding value in the filter.
- 2.Add up the results to get a single number.

This number is the **output** for that location, and it represents how much the filter detects that feature in the image.

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

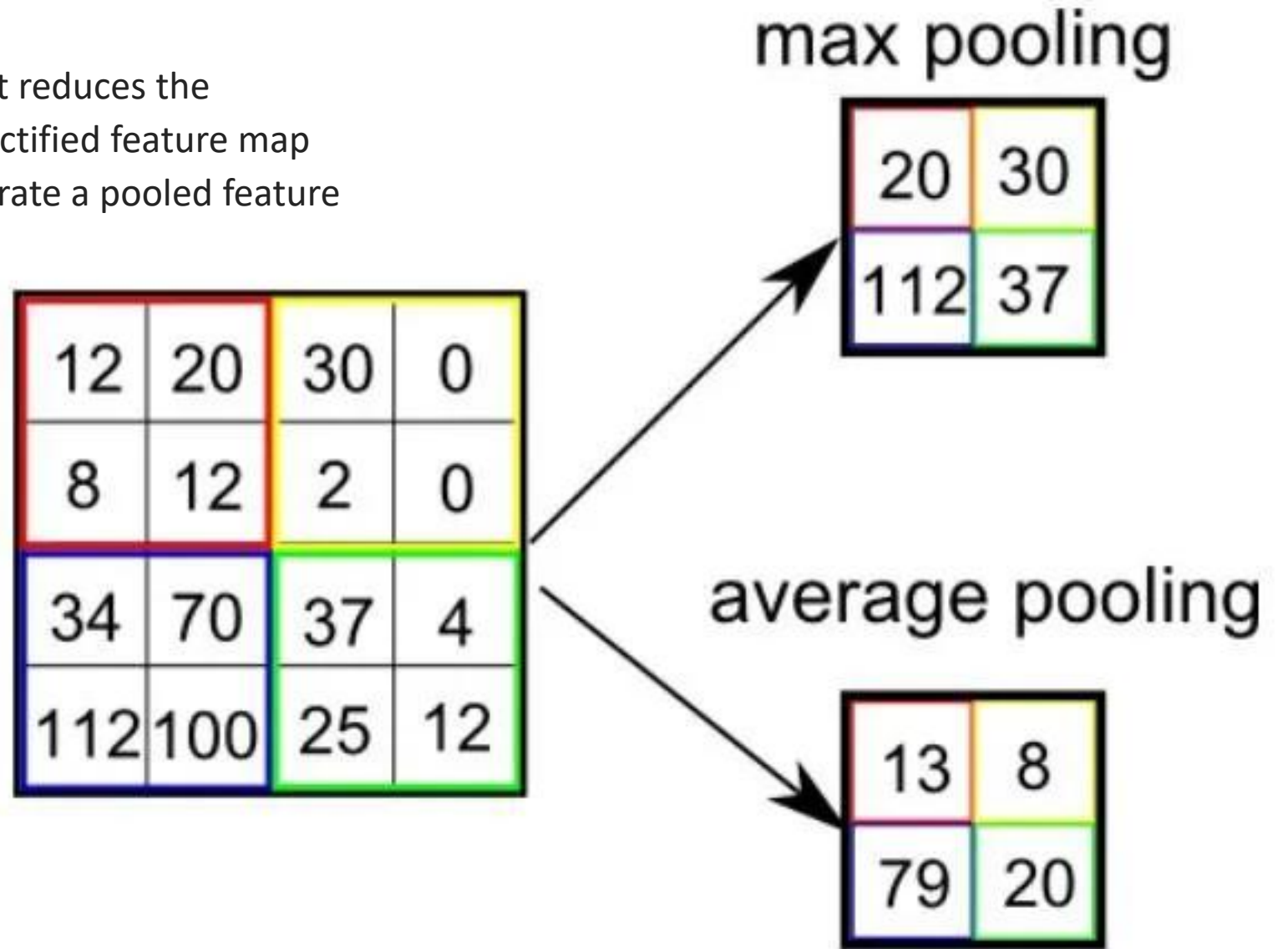
=

6		

$$\begin{aligned}
 &7 \times 1 + 4 \times 1 + 3 \times 1 + \\
 &2 \times 0 + 5 \times 0 + 3 \times 0 + \\
 &3 \times -1 + 3 \times -1 + 2 \times -1 \\
 &= 6
 \end{aligned}$$

2. Pooling Layer

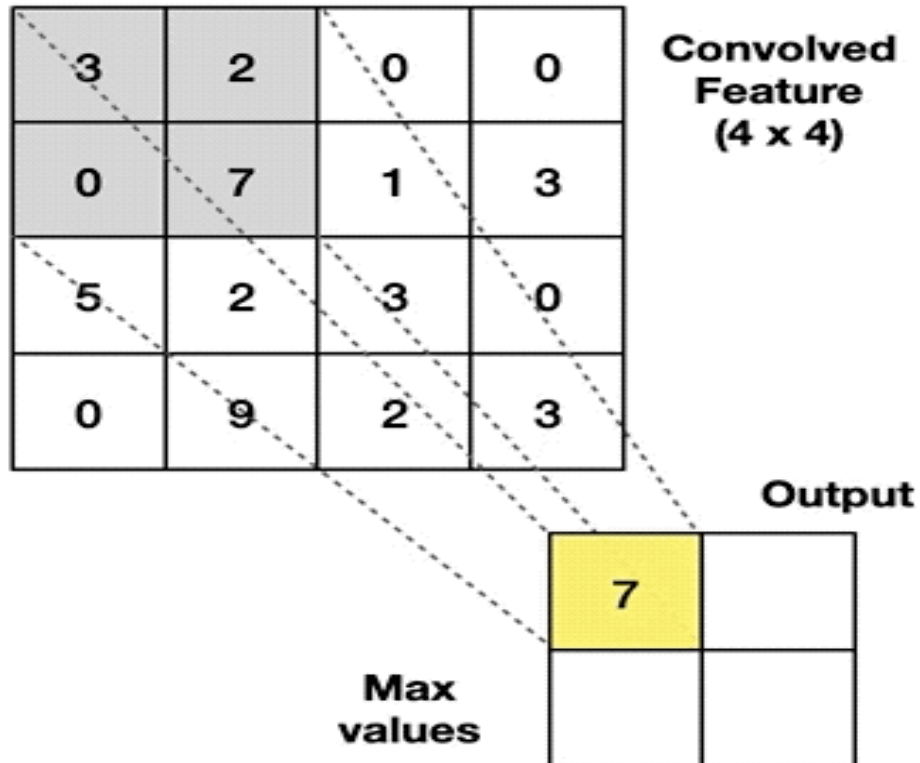
Pooling is a down-sampling operation that reduces the dimensionality of the feature map. The rectified feature map now goes through a pooling layer to generate a pooled feature map.



Max Pooling

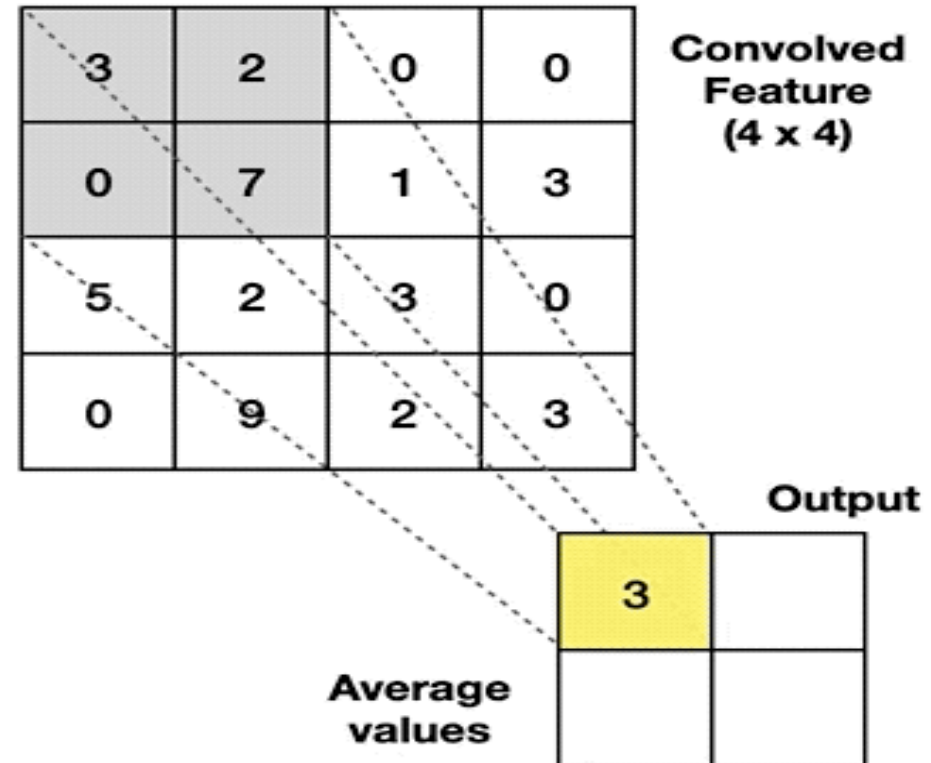
Take the **highest** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

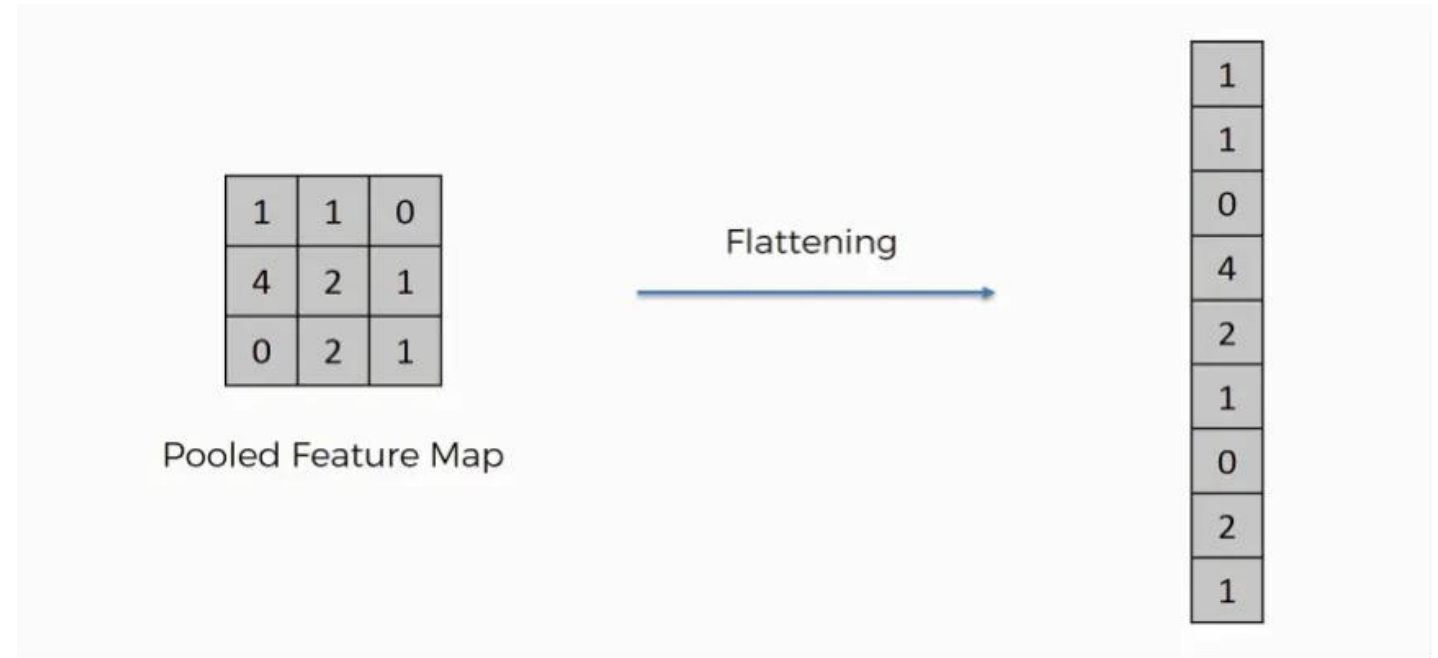
Calculate the **average** value from the area covered by the kernel



There are two types of Pooling: **Max Pooling** and **Average Pooling**.

- **Max Pooling** returns the maximum value from the portion of the image covered by the Kernel
- **Average Pooling** returns the average of all the values from the portion of the image covered by the Kernel.

3. Fully Connected Input Layer (Flatten)



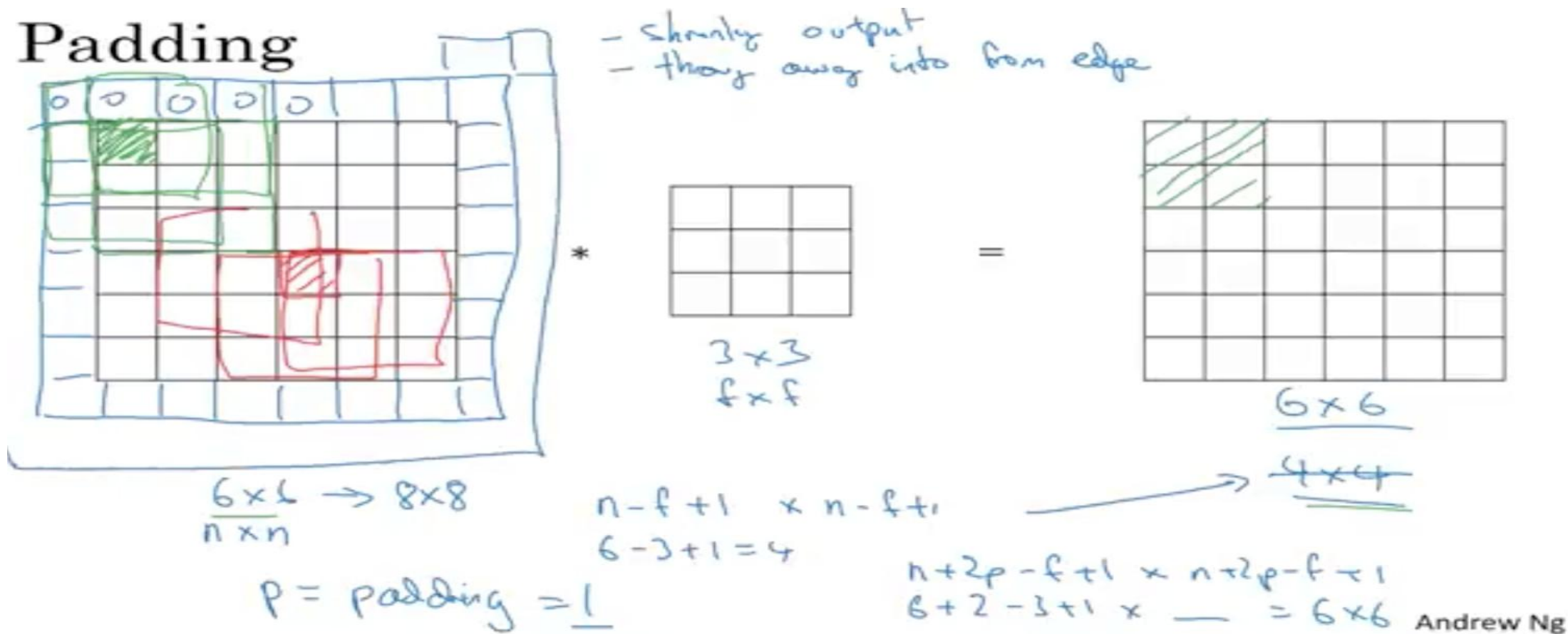
- Fully connected layers link each neuron in one layer to every neuron in the next, functioning similarly to a traditional multi-layer perceptron (MLP) network.
- Fully connected layers are layers where all the inputs from one layer are connected to every activation unit of the next layer. The layer takes the output of the pooling and flatten them into single vector.
- The purpose of this layer is to classify the image into a label. It takes the output of previous layer and predicts the best label by applying weights and “voting”. The final output will be the probabilities for each label.

Padding and Strided Convolution

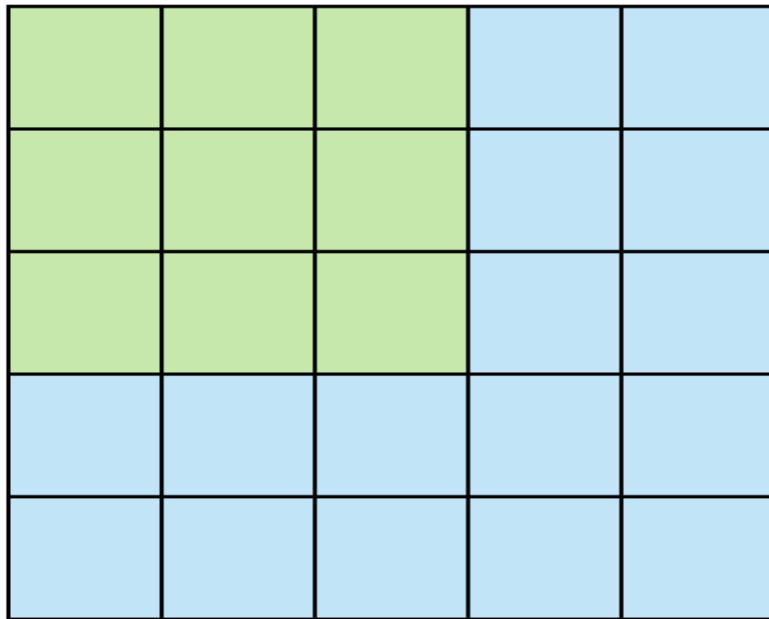
- In order to build deep neural networks, one modification to the basic convolutional operation that you need to really use is padding.
- A 6x6 input image and convolve it with a 3x3 filter, you end up with a 4x4 output image.
- And the generic map is if you have an $n \times n$ input image and convolve it with an $f \times f$ filter you'll get an $(n-f+1) \times (n-f+1)$ output image.

There are two downsides to this map, one is that if everytime you do apply convolutional operator your image shrinks, you can do this few times before your image starts getting really small. The second downside is if you look at the pixel at the corner of that input image, these pixels are touched and used only in few 3x3 regions, particularly the upleft corner pixel is touched in only one 3x3 region, whereas if you take pixel in the middle of the input image then there are a lot of 3x3 regions that overlap that pixel, so it's as if pixel on the corners around the edges of input image is used much less in the output so you're throwing away a lot of the information near the edge of the input image.

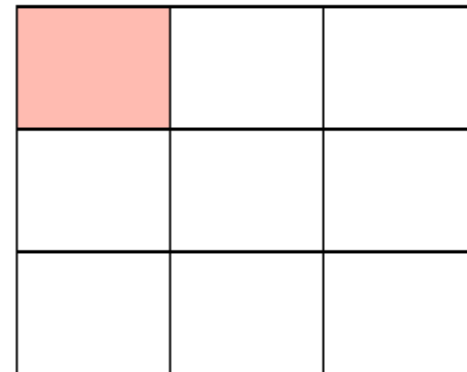
So, in order to fix both of these problems, what you can do is before applying convolutional operation you can pad the image, so in the following case, you can pad the input image with an additional border of one pixel around the edges, then instead of 6x6 image you've now padded this to an 8x8 image, and if you convolve an 8x8 image with a 3x3 filter you now get not the 4x4 but a 6x6 output image. So, you've managed to preserve the original input size of 6x6.



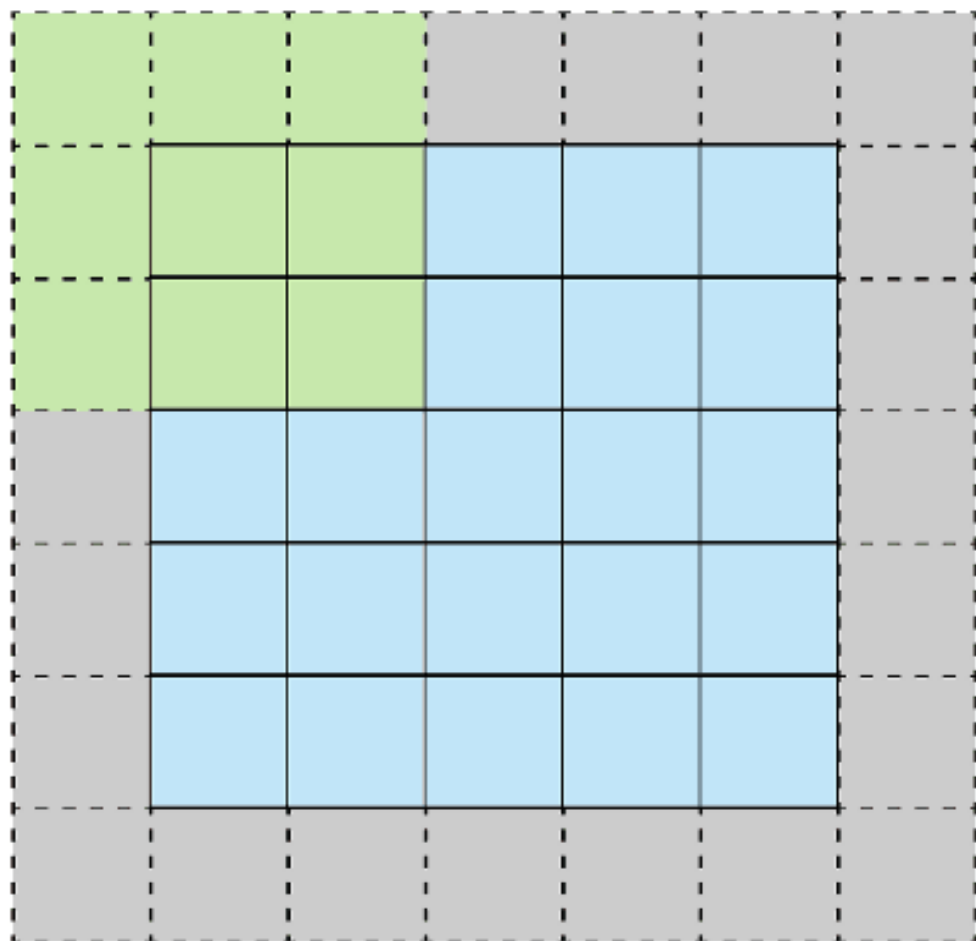
- **Stride** denotes how many steps we are moving in each step in convolution. By default, it is one. We can observe that the size of output is smaller than input. To maintain the dimension of output as in input, we use padding.
- **Padding** is a process of adding zeros to the input matrix symmetrically. In the following example, the extra grey blocks denote the padding. It is used to make the dimension of output same as input.



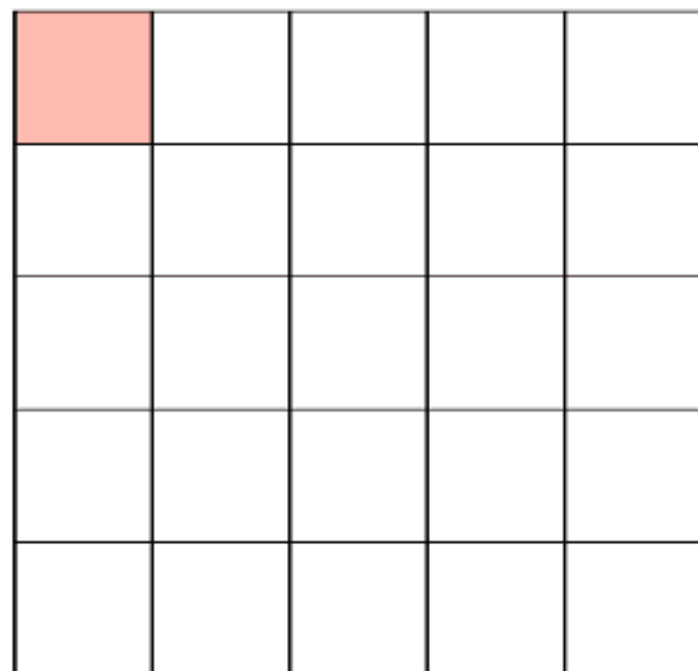
Stride 1



Feature Map



Stride 1 with Padding



Feature Map

Channels

In images, **channels** represent different layers of information. A grayscale (black-and-white) image has just one channel, where each pixel holds a single brightness value. However, in a **color image** (like most photos), we need three channels to capture the colors: **Red, Green, and Blue (RGB)**. Each color channel is like a separate layer of the image, and together they form the final-colored picture.

So, for a **28 x 28** grayscale image, the data is **28 x 28 x 1**, since it has only one layer. But for a **28 x 28** color image, the data is **28 x 28 x 3** because it has three channels (RGB).

Example with Convolution

- 1.Input Channels:** Let’s say you start with a **100 x 100 x 3** image (where 3 is for RGB channels).
- 2.Filters (or Kernels):** The convolutional layer might have several filters, each designed to detect a specific feature. For instance, if you have 10 filters, each will scan across the image.
- 3.Output Channels:** Each filter creates its own output map. So, if you use 10 filters, the convolutional layer’s output will be **100 x 100 x 10**.

In this way, the **depth of the image data** (i.e., the number of channels) increases, as each filter layer adds its own new channel. This allows CNNs to detect and store multiple features from the image in different “layers” or “channels” of information.

Strided convolution

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3 ³	4 ⁴	6 ⁴
3	2	4	1	9 ¹	8 ⁰	3 ²
0	1	3	9	2 ⁻¹	1 ⁰	4 ³

7x7

*

3	4	4
1	0	2
-1	0	3

3x3

=

91	100	83
69	91	127
44	72	74

3x3

Stride = 2

$\lfloor z \rfloor = \text{floor}(z)$

$n \times n$ * $f \times f$
 padding p stride s
 $s = 2$

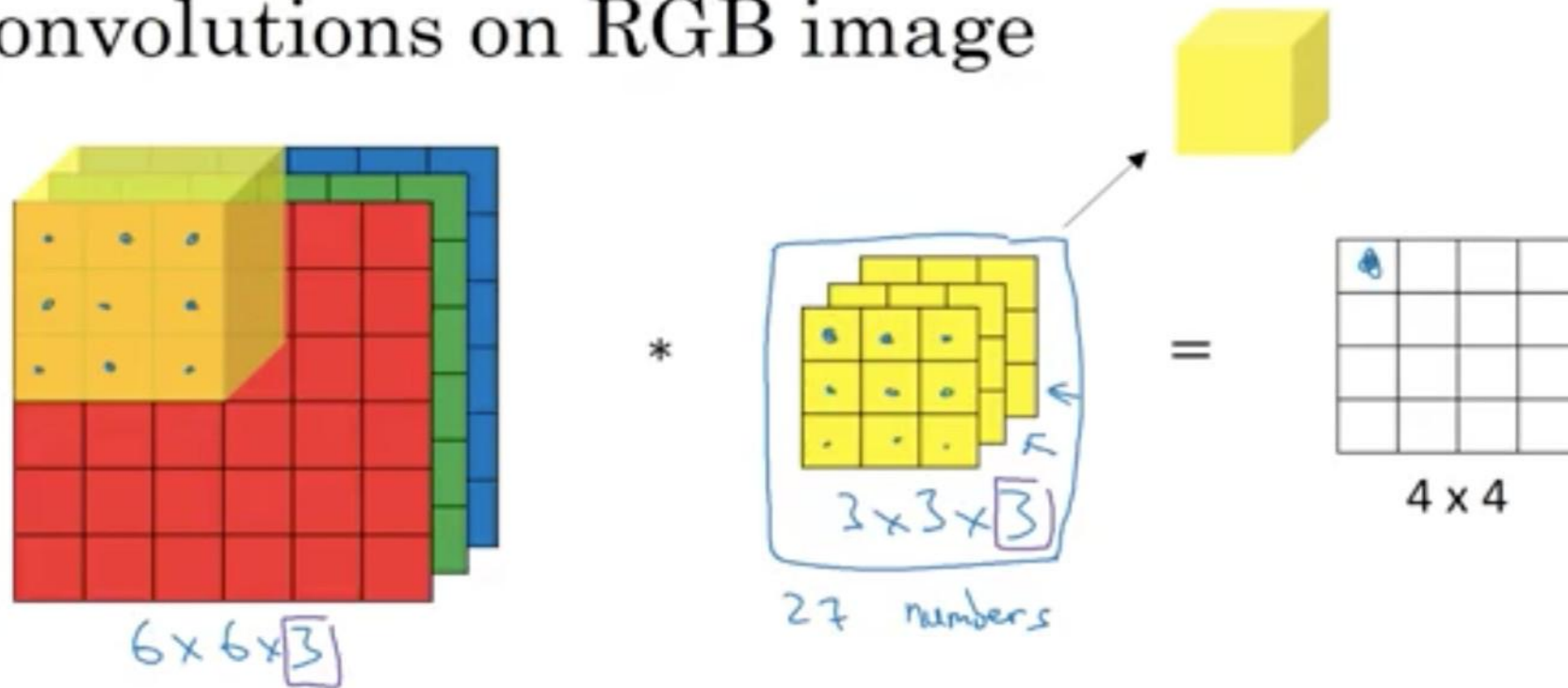
$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

$$\frac{7 + 0 - 3}{2} + 1 = \frac{4}{2} + 1 = 3$$

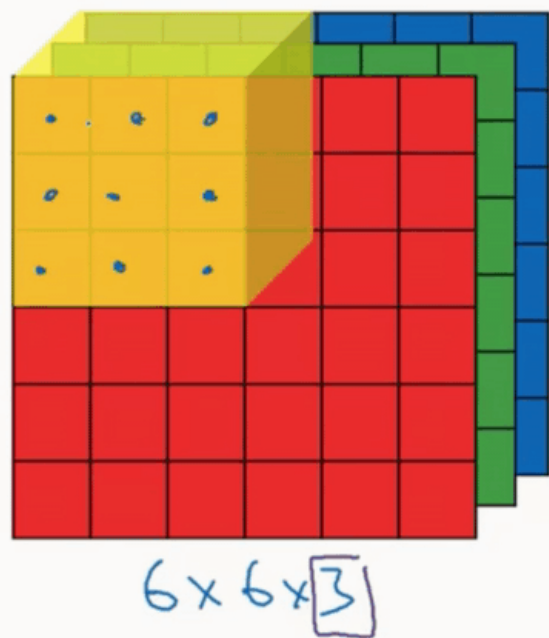
Andrew Ng

Convolutions over volumes,

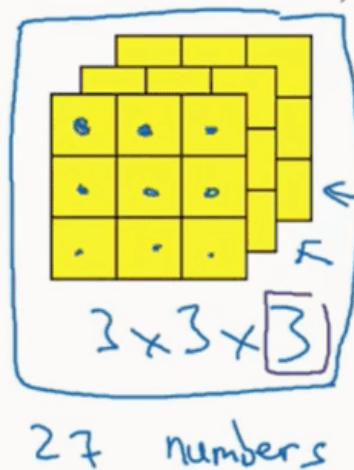
Convolutions on RGB image



Convolutions on RGB image



*

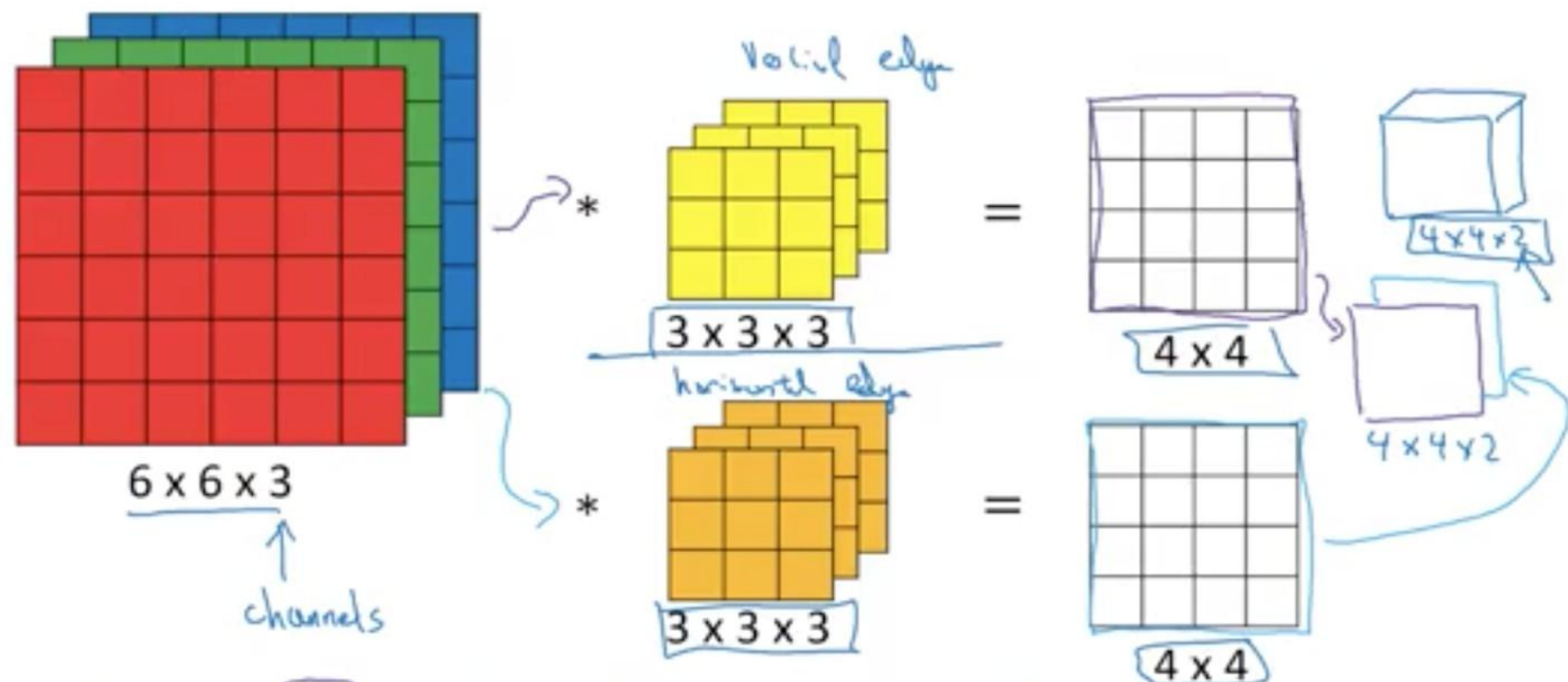


=



Now, let's say that there's a second 3 by 3 by 3 filter denoted by an orange color, is a horizontal edge detector as shown of *figure 4*. So, convolving the image with the the yellow filter gives you a 4 by 4 output and convolving with the orange filter gives you a different 4 by 4 output. Now, what we can do is then take these two 4 by 4 outputs and stack them to get a 4 by 4 by 2 output. Notice that if the image was first convolved with the yellow filter , then we take the image & yellow filter's output to be the first one (at the front) and you can then take the image & orange second filter's output and stack it at the back to end up with the 4 by 4 by 2 output image shown on the right in *figure 4*. Notice that the 2 comes from the fact that we used two different filters.

Multiple filters



Summary: $n \times n \times n_c \times f \times f \times n_c \rightarrow \frac{n-f+1}{4} \times \frac{n-f+1}{4} \times n'_c$

$6 \times 6 \times 3 \times 3 \times 3 \times 3 \rightarrow 4 \times 4 \times 2$ (where $n'_c = 2$ is the number of filters)

Soft max regression

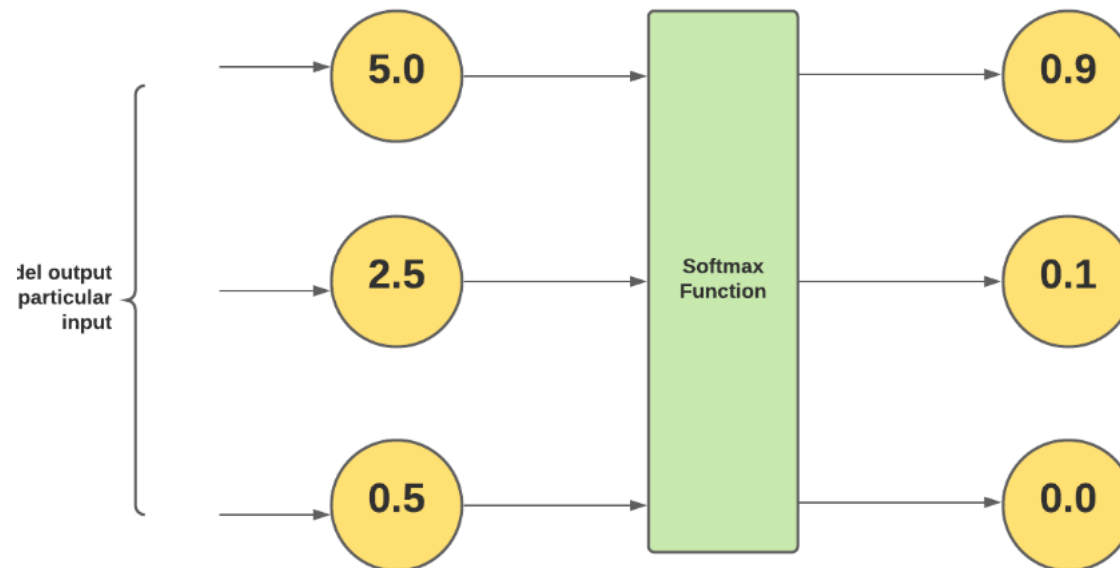
In softmax regression sum of all probabilities is equal to 1.

Definition

The Softmax regression is a form of logistic regression that normalizes an input value into a vector of values that follows a probability distribution whose total sums up to 1. The output values are between the range $[0,1]$ which is nice because we are able to avoid binary classification and accommodate as many classes or dimensions in our neural network model.

Example:

- Let's understand this with an example. Let's say the models (such as those trained using algorithms such as multi-class or multinomial logistic regression) output three different values such as 5.0, 2.5, and 0.5 for a particular input.
- In order to convert these numbers into probabilities, these numbers are fed into the softmax function.



Observation:

- Notice that the softmax outputs are less than 1. And, the outputs of the softmax function sum up to 1. This property, the Softmax function is considered an activation function in neural networks and algorithms such as multinomial logistic regression. Note that for binary logistic regression, the activation function used is the sigmoid function.
- Based on the above, it could be understood that the output of the softmax function maps to a $[0, 1]$ range. And, it maps outputs in a way that the total sum of all the output values is 1. Thus, it could be said that the output of the softmax function is a **probability distribution**.

Deep Learning Frameworks:

1. TensorFlow

[TensorFlow](#) is one of the most popular, open-source libraries that is being heavily used for ***numerical computation deep learning***. Google introduced it in 2015 for their internal RnD work but later when they saw the capabilities of this framework, they decided to make it open and the repository is available at [TensorFlow Repository](#). As you'll see, learning deep learning is pretty complex but making certain implementations are far easy and by such frameworks, it's even smooth to process the desired outcomes.

How Does it Work?

This framework allows you to create *dataflow graphs and structures* to specify **how data travels through a graph** with the help of inputs as tensors (also known as a multi-dimensional graph). Tensor Flow allows users to prepare a flowchart and based on their inputs, it generates the output.

Applications of Tensor Flow:

- **Text-Based Application:** Nowadays text-based apps are being heavily used in the market that including language detection, sentimental analysis (for social media to block abusive posts)
- **Image Recognition (I-R) Based System:** Today most sectors have introduced this technology in their system for motion, facial and photo-clustering models.
- **Video Detection:** Real-time object detection is a computer vision technique to detect the motion (from both image and video) to trace back any object from the provided data.

2. PyTorch

The most famous, that even powers “*Tesla Auto-Pilot*” is none other than [Pytorch](#) which works on deep learning technology. It was first introduced in 2016 by a group of people (Adam Paszke, Sam Gross, Soumith Chintala, and Gregory Chanan), under Facebook’s AI lab. The interesting part about PyTorch is that both [C++](#) & [Python](#) can use it but python’s interface is the most polished. Not so surprising, Pytorch is being backed by some of the top giants in the tech industry (Google, Salesforce, Uber, etc.). It was introduced to achieve two major goals, the first is to remove the requirement of NumPy (so that it can power GPU with tensor) and the second is to offer an automatic differentiation library (that is useful to implement neural networks).

How Does it Work?

This framework uses a computational dynamic graph right after the declaration of variables. Besides this, it uses Python’s basic concepts like loops, structures, etc. We have often used NLP functions in our smartphones (such as Apple’s Siri or Google Assistant), they all use deep learning algorithms known as RNN or Recurrent Neural Network.

Applications of PyTorch:

- **Weather Forecast:** To predict and highlight the pattern of a particular set of data, Pytorch is being used (not only for forecast but also for real-time analysis).
- **Text Auto Detection:** We might have noticed sometimes whenever we try to search something on Google or any other search engine, it starts showing “auto-suggestion” and that’s where the algorithm works and Pytorch is being used
- **Fraud Detection:** To prevent any unauthorized activities on credit/debit cards, this algorithm is being used to apply anomalous behavior and outliers.

1. Theano

To *define any mathematical expressions in deep learning*, we use Python's library Theano. It was named after a great greek mathematician "[Theano](#)". It was released in 2007 by MILA (Montreal Institute for Learning Algorithms) and Theano uses a host of clever code optimizations to deliver as much performance at maximum caliber from your hardware. Besides this, there are two salient features are at the core of any deep learning library:

- The tensor operations, and
- The capability to run the code on CPU or Graphical Computation Unit (GPU).

These two features enable us to work with a big bucket of data. Moreover, Theano proposes automatic differentiation which is a very useful feature and can also solve numeric optimization on a big picture than deep learning complex issues.

How Does it Work?

If you talk about its working algorithm, *Theano itself is effectively dead, but the deep learning frameworks built on top of Theano, are still functioning* which also include the more user-friendly frameworks- *Keras, Lasagne, and Blocks* that offer a high-level framework for fast prototyping and model testing in deep learning and machine learning algorithms.

Applications of Theano:

- **Implementation Cycle:** Theanos works in 3 different steps where it starts by defining the objects/variables then moves into different stages to define the mathematical expressions (in the form of functions) and at last it helps in evaluating expressions by passing values to it.
- Companies like IBM are using Theanos for implementing neural networks and to enhance their efficiency
- For using Theanos, make sure you have pre-installed some of the following dependencies: Python, NumPy, SciPy, and BLAS (for matrix operations).

1. Keras

Since we've been talking about deep learning and the complexity it has, **Keras** is another library that is highly productive and dedicatedly focuses on solving deep learning problems. Besides this, Keras also help engineers to take full advantage of the scalability and cross-platform capabilities to apply within their projects. It was first introduced in 2015 under ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System) project. Keras is an open-source platform and is being actively used as a part of python's interface in machine learning and deep neural learning. Today, big tech giants like Netflix, Uber, etc. are using Keras actively to improve their scalability.

How Does it Work?

The architecture of Keras has been designed in such a way that it acts as a high-level neural network (written in Python). Besides this, It works as a wrapper for low-level libraries (*such as TensorFlow or Theano*) and high-level neural network libraries. It was introduced with the concept to perform fast testing and experiment before going on the full scale.

Applications of Keras:

- Today, companies are using Keras to develop smartphones powered by machine learning and deep learning in their system. Apple company is one of the biggest giants that has incorporated this technology in past few years.
- In the healthcare industry, developers have built a predictive technology where the machine can predict the patient's diagnosis and can also alert pre-heart attack issues. (Thus, this machine can predict the chances of detecting heart disease, based on provided data).
- Face Mask Detection: During the pandemic, many companies have offered various contributions and companies have built a system using deep learning mechanisms for using facial recognition to detect whether the person is wearing a facial mask or not. (Nokia was among one the companies to initiate this using the Keras library)

Train, Validation and Test Sets

Training Dataset

Training Dataset: The sample of data used to fit the model.

The actual dataset that we use to train the model (weights and biases in the case of a Neural Network). The model sees and learns from this data.

Validation Dataset

Validation Dataset: The sample of data used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters. The evaluation becomes more biased as skill on the validation dataset is incorporated into the model configuration.

The validation set is used to evaluate a given model, but this is for frequent evaluation. We, as machine learning engineers, use this data to fine-tune the model hyperparameters. Hence the model occasionally sees this data, but never does it “Learn” from this. We use the validation set results, and update higher level hyperparameters. So the validation set affects a model, but only indirectly. The validation set is also known as the Dev set or the Development set. This makes sense since this dataset helps during the “development” stage of the model.

Test Dataset

Test Dataset: *The sample of data used to provide an unbiased evaluation of a final model fit on the training dataset.*

The Test dataset provides the gold standard used to evaluate the model. It is only used once a model is completely trained(using the train and validation sets). The test set is generally what is used to evaluate competing models (For example on many Kaggle competitions, the validation set is released initially along with the training set and the actual test set is only released when the competition is about to close, and it is the result of the the model on the Test set that decides the winner). Many a times the validation set is used as the test set, but it is not good practice. The test set is generally well curated. It contains carefully sampled data that spans the various classes that the model would face, when used in the real world.

About the dataset split ratio

Now that you know what these datasets do, you might be looking for recommendations on how to split your dataset into Train, Validation and Test sets.

This mainly depends on 2 things. First, the total number of samples in your data and second, on the actual model you are training.

Some models need substantial data to train upon, so in this case you would optimize for the larger training sets. Models with very few hyperparameters will be easy to validate and tune, so you can probably reduce the size of your validation set, but if your model has many hyperparameters, you would want to have a large validation set as well(although you should also consider cross validation). Also, if you happen to have a model with no hyperparameters or ones that cannot be easily tuned, you probably don't need a validation set too!

Bias Variance trade off

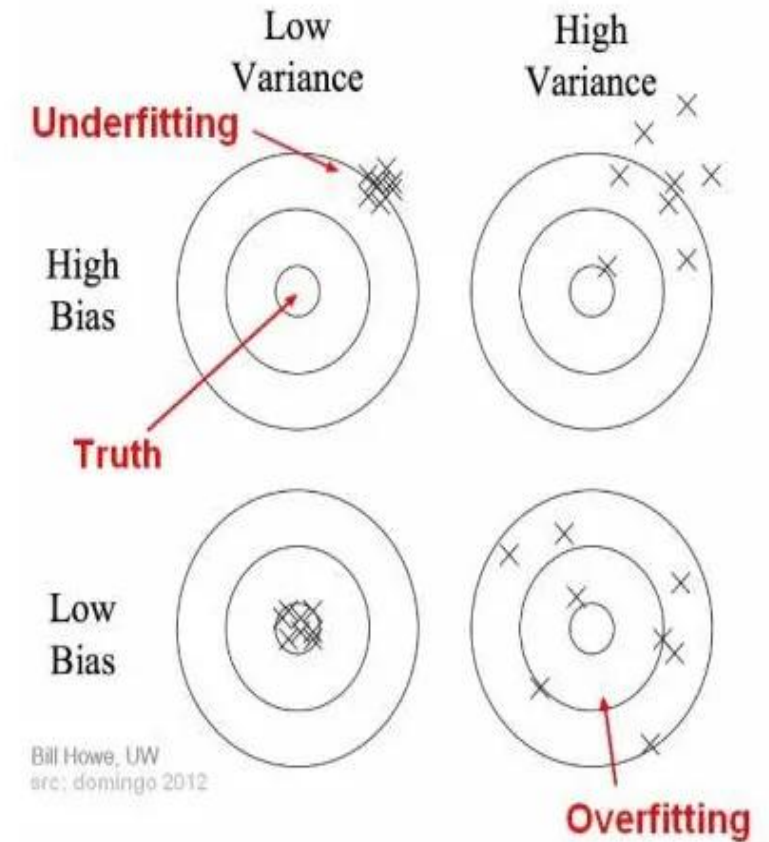
What is variance?

Variance is the variability of model prediction for a given data point or a value which tells us spread of our data. Model with high variance pays a lot of attention to training data and does not generalize on the data which it hasn't seen before. As a result, such models perform very well on training data but has high error rates on test data.

What is bias?

Bias is the difference between the average prediction of our model and the correct value which we are trying to predict. Model with high bias pays very little attention to the training data and oversimplifies the model. It always leads to high error on training and test data.

- In the above diagram, center of the target is a model that perfectly predicts correct values. As we move away from the bulls-eye our predictions become get worse and worse. We can repeat our process of model building to get separate hits on the target.
- In supervised learning, **underfitting** happens when a model unable to capture the underlying pattern of the data. These models usually have **high bias and low variance**. It happens when we have very less amount of data to build an accurate model. Also, these kind of models are very simple to capture the complex patterns in data like Linear and logistic regression.
- In supervised learning, **overfitting** happens when our model captures the noise along with the underlying pattern in data. It happens when we train our model a lot over noisy dataset. These models have **low bias and high variance**. These models are very complex like Decision trees which are prone to overfitting.



Why is Bias Variance Tradeoff?

If our model is too simple and has very few parameters then it may have **high bias and low variance**. On the other hand if our model has large number of parameters then it's going to have **high variance and low bias**. So we need to find the right/good balance without overfitting and underfitting the data.

This tradeoff in complexity is why there is a tradeoff between bias and variance. An algorithm can't be more complex and less complex at the same time.

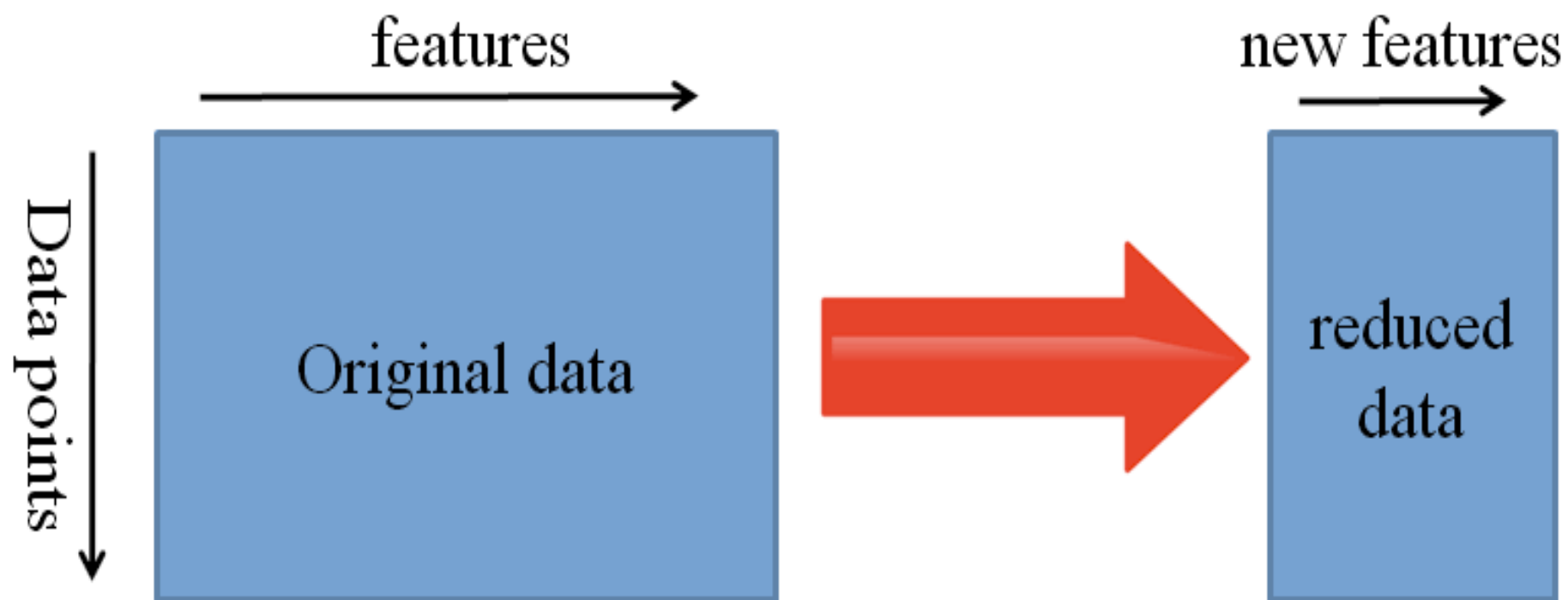
To build a good model, we need to find a good balance between bias and variance such that it minimizes the total error.

What Is Transfer Learning?

Transfer learning, used in machine learning, is the reuse of a pre-trained model on a new problem. In transfer learning, a machine exploits the knowledge gained from a previous task to improve generalization about another. For example, in training a classifier to predict whether an image contains food, you could use the knowledge it gained during training to recognize drinks.

In transfer learning, the knowledge of an already trained [machine learning](#) model is applied to a different but related problem.

- With transfer learning, we basically try to exploit what has been learned in one task to improve generalization in another.
- We transfer the weights that a network has learned at “task A” to a new “task B.”
- The general idea is to use the knowledge a model has learned from a task with a lot of available [labeled training data](#) in a new task that doesn’t have much data.
- Instead of starting the learning process from scratch, we start with patterns learned from solving a related task.
- Transfer learning is mostly used in computer vision and [natural language processing](#) tasks like sentiment analysis due to the huge amount of computational power required.



In [computer vision](#), for example, neural networks usually try to detect edges in the earlier layers, shapes in the middle layer and some task-specific features in the later layers.

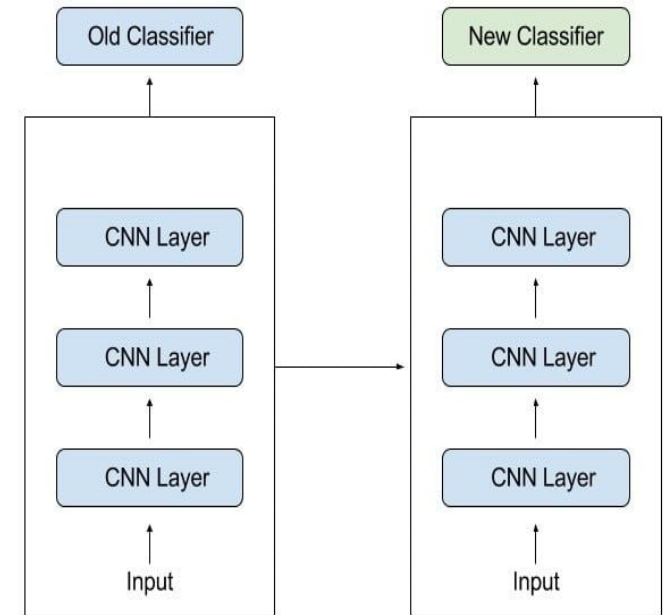
In transfer learning, the early and middle layers are used and we only retrain the latter layers. It helps leverage the labeled data of the task it was initially trained on. This process of retraining models is known as **fine-tuning**.

In the case of transfer learning, though, we need to isolate specific layers for retraining. There are then two types of layers to keep in mind when applying transfer learning:

- **Frozen layers:** Layers that are left alone during retraining and keep their knowledge from a previous task for the model to build on.
- **Modifiable layers:** Layers that are retrained during fine-tuning, so a model can adjust its knowledge to a new, related task.

Why Use Transfer Learning

- The main advantages of transfer learning are saving training time, improving the performance of neural networks (in most cases) and not needing a lot of data.
- Usually, a lot of data is needed to train a neural network from scratch, but access to that data isn't always available.
- With transfer learning, a solid [machine learning model](#) can be built with comparatively little training data because the model is already pre-trained.
- This is especially valuable in natural language processing because mostly expert knowledge is required to create large labeled [data sets](#).
- Additionally, training time is reduced because it can sometimes take days or even weeks to train a deep neural network from scratch on a complex task.



When to Use Transfer Learning

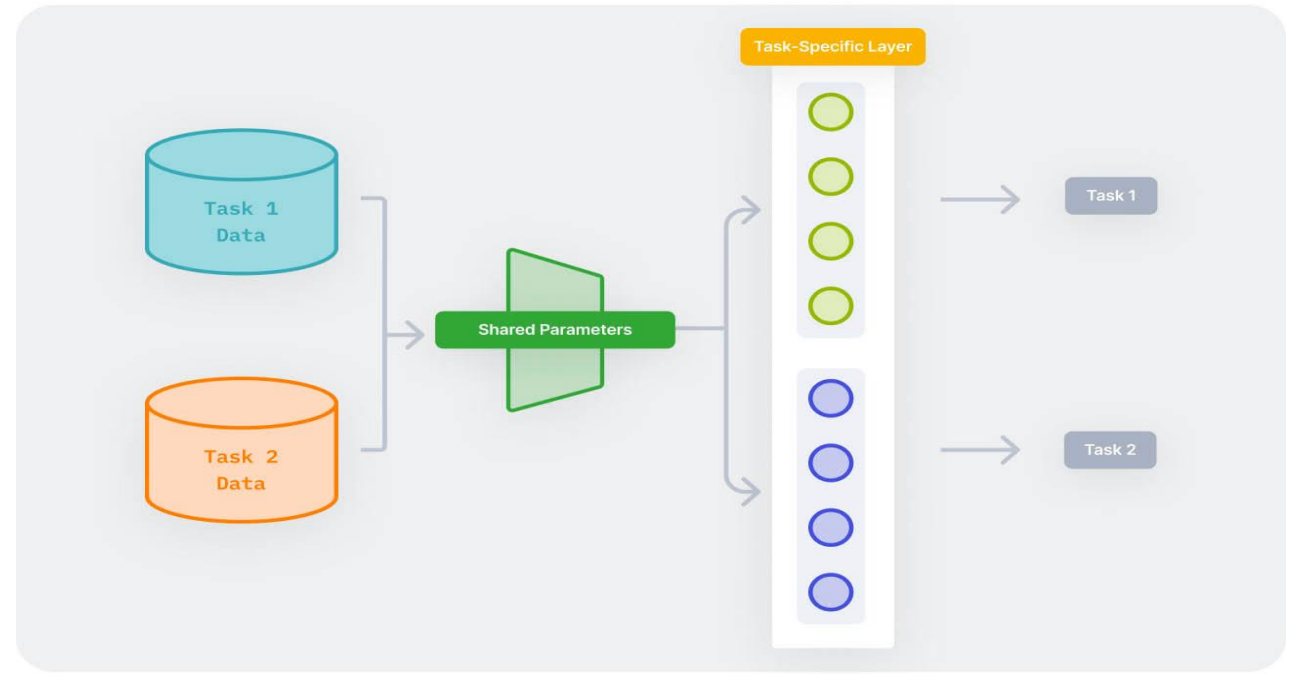
As is always the case in [machine learning](#), it is hard to form rules that are generally applicable, but here are some guidelines on when transfer learning might be used:

- Lack of training data:** There isn't enough labeled training data to train your network from scratch.
- Existing network:** There already exists a network that is pre-trained on a similar task, which is usually trained on massive amounts of data.
- Same input:** When task 1 and task 2 have the same input.

If the original model was trained using an [open-source library](#) like TensorFlow, you can simply restore it and retrain some layers for your task.

Keep in mind, however, that transfer learning only works if the features learned from the first task are general, meaning they can be useful for another related task as well. Also, the input of the model needs to have the same size as it was initially trained with. If you don't have that, add a pre-processing step to resize your input to the needed size.

Multi-Task Learning



Multi-Task Learning (MTL) is a type of machine learning technique where a model is trained to perform multiple tasks simultaneously. In deep learning, MTL refers to training a neural network to perform multiple tasks by sharing some of the network's layers and parameters across tasks.

In MTL, the goal is to improve the generalization performance of the model by leveraging the information shared across tasks. By sharing some of the network's parameters, the model can learn a more efficient and compact representation of the data, which can be beneficial when the tasks are related or have some commonalities.

- There are different ways to implement MTL in deep learning, but the most common approach is to use a **shared feature extractor** and multiple task-specific heads.
- The shared feature extractor is a part of the network that is shared across tasks and is used to extract features from the input data. The task-specific heads are used to make predictions for each task and are typically connected to the shared feature extractor.
- Another approach is to use a **shared decision-making layer**, where the decision-making layer is shared across tasks, and the task-specific layers are connected to the shared decision-making layer.
- MTL can be useful in many applications such as natural language processing, computer vision, and healthcare, where multiple tasks are related or have some commonalities. It is also useful when the data is limited, MTL can help to improve the generalization performance of the model by leveraging the information shared across tasks.

What is Multi-Task Learning? Multi-Task learning is a sub-field of Machine Learning that aims to solve multiple different tasks at the same time, by taking advantage of the similarities between different tasks. This can improve the learning efficiency and also act as a regularizer which we will discuss in a while.

Intuition behind Multi-Task Learning (MTL): By using Deep learning models, we usually aim to learn a good representation of the features or attributes of the input data to predict a specific value. Formally, we aim to optimize for a particular function by training a model and fine-tuning the hyperparameters till the performance can't be increased further. By using MTL, it might be possible to increase performance even further by forcing the model to learn a more generalized representation as it learns (updates its weights) not just for one specific task but a bunch of tasks. Biologically, humans learn in the same way. We learn better if we learn multiple related tasks instead of focusing on one specific task for a long time.

Assumptions and Considerations – Using MTL to share knowledge among tasks are very useful only when the tasks are very similar, but when this assumption is violated, the performance will significantly decline. **Applications:** MTL techniques have found various uses, some of the major applications are-

- Object detection and Facial recognition
- Self Driving Cars: Pedestrians, stop signs and other obstacles can be detected together
- Multi-domain collaborative filtering for web applications
- Stock Prediction
- Language Modelling and other NLP applications